

# SIGEC-PJ

## Sistema de Gestión de Equipos CCTV del Poder Judicial

*Documentación técnica profesional del proyecto*

---

Arquitectura · Diseño de base de datos · Operaciones · Seguridad · Despliegue · Evidencia de alta  
disponibilidad (99 %)

---

### **Base de Datos II — Sistema de Alta Disponibilidad**

Universidad Politécnica Internacional — Licenciatura en Informática

Bolaños Dixon Luis Felipe · Molina Monge Juan Pablo  
Castillo Kieth Joshua · Sanche Gámez Gustavo Adolfo

Profesor: Julio César Sánchez Lobo

Fecha: 2 de julio de 2026

## Contenido

---

|                                                          |    |
|----------------------------------------------------------|----|
| Contenido.....                                           | 2  |
| 1. Resumen del proyecto.....                             | 3  |
| 1.1 Objetivos.....                                       | 3  |
| 1.2 Alcance por fases.....                               | 3  |
| 2. Arquitectura general.....                             | 3  |
| 2.1 Flujo de una petición.....                           | 4  |
| 3. Componentes del stack.....                            | 4  |
| 4. Diseño de la base de datos.....                       | 6  |
| 4.1 Modelo entidad-relación.....                         | 6  |
| 4.2 Diccionario de datos .....                           | 6  |
| 4.3 Integridad y automatización .....                    | 9  |
| 4.4 Extracto del DDL (entidad central y auditoría) ..... | 9  |
| 5. Operaciones del sistema.....                          | 11 |
| 5.6 Ciclo de vida ITIL de un incidente .....             | 12 |
| 5.7 Reglas de negocio clave .....                        | 13 |
| 6. Seguridad.....                                        | 13 |
| 7. Guía de despliegue en local .....                     | 14 |
| 7.1 Requisitos previos .....                             | 14 |
| 7.2 Variables de entorno.....                            | 14 |
| 7.3 Levantar y verificar el stack.....                   | 15 |
| 7.4 Acceso a la aplicación.....                          | 15 |
| 7.5 Problemas comunes .....                              | 15 |
| 8. Correcciones aplicadas durante la verificación .....  | 16 |
| 9. Evidencia de pruebas ejecutadas (Fase 3).....         | 17 |
| 9.1 Arranque del clúster HA.....                         | 17 |
| 9.2 Autenticación y API REST.....                        | 17 |
| 9.3 Prueba de failover (RTO / RPO) .....                 | 17 |
| 9.4 Estado final de los servicios.....                   | 18 |
| 10. Estado final y checklist de aceptación .....         | 19 |

## 1. Resumen del proyecto

SIGEC-PJ digitaliza la gestión del inventario, los incidentes y el ciclo de vida de los equipos CCTV del Poder Judicial de Costa Rica. Reemplaza una hoja de cálculo ODS —sin integridad referencial y con credenciales en texto plano— por una base de datos PostgreSQL 16 normalizada hasta la tercera forma normal (14 entidades de negocio + auditoría), un backend Java 21 con Spring Boot 3 y un frontend Vue.js 3, desplegados sobre una topología de alta disponibilidad que cumple el objetivo del curso: 99 % de disponibilidad, con RPO = 0 y RTO < 60 segundos.

### 1.1 Objetivos

- Eliminar el almacenamiento de credenciales en texto plano y garantizar la integridad referencial de los datos.
- Ofrecer una API REST segura con control de acceso por rol y trazabilidad completa mediante auditoría automática.
- Sostener el servicio ante la caída de un nodo de base de datos sin pérdida de datos ni intervención manual.

### 1.2 Alcance por fases

| Fase   | Contenido                                                                        | Estado                  |
|--------|----------------------------------------------------------------------------------|-------------------------|
| Fase 1 | Diseño: situación actual, modelo ER, riesgos y propuesta de alta disponibilidad. | Completada              |
| Fase 2 | Migración ODS → PostgreSQL, API REST, frontend y análisis costo/beneficio.       | Completada              |
| Fase 3 | Clúster HA, failover medido, guías operativas y demostración end-to-end.         | Completada y verificada |

## 2. Arquitectura general

El sistema se compone de tres capas redundantes —presentación (SPA Vue.js servida por nginx), aplicación (dos instancias Spring Boot activo-activo) y datos (clúster PostgreSQL con failover automático)— coordinadas por HAProxy como punto único de entrada con terminación TLS 1.3.

## 2.1 Flujo de una petición

### 3. Componentes del stack

Página 4 de 19

| Componente                       | Función                                                                                                                                                                                                                                                                                                                               |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Patroni + PostgreSQL 16          | Cada nodo corre PostgreSQL 16 gestionado por un agente Patroni con replicación síncrona ( <code>synchronous_mode: true</code> ): ninguna transacción se confirma sin estar escrita en la réplica → RPO = 0. Al perder el primario, promueve la réplica sincronizada en segundos y expone el rol de cada nodo por su API REST (:8008). |
| HAProxy                          | Puerto 5000: escritura al primario; 5001: lectura balanceada entre réplicas; 8443: HTTPS/TLS 1.3 con enrutamiento por path; 7000: panel de estadísticas.                                                                                                                                                                              |
| pgBackRest                       | Backups completos y WAL archiving continuo, habilitando recuperación punto-en-el-tiempo (PITR). Independiente del clonado de réplicas (basebackup nativo para el bootstrap).                                                                                                                                                          |
| Backend (Spring Boot 3, Java 21) | Arquitectura en capas controlador → servicio → repositorio → entidad (JPA/Hibernate). Dos instancias idénticas activo-activo.                                                                                                                                                                                                         |
| Frontend (Vue.js 3 + nginx)      | SPA compilada a estático y servida por nginx; consume la API por el mismo dominio HTTPS de HAProxy, evitando CORS.                                                                                                                                                                                                                    |

## 4. Diseño de la base de datos

Motor: PostgreSQL 16. Base de datos: sigecpj. Rol de aplicación: sigec (permisos acotados; el superusuario postgres se reserva para administración del clúster). Modelo relacional normalizado hasta 3FN: 14 entidades de negocio + 1 tabla de auditoría + 1 vista.

### 4.1 Modelo entidad-relación

El diagrama utiliza notación crow's foot: la clave primaria (PK) y las claves foráneas (FK) se indican en cada entidad, y cada relación muestra su cardinalidad (uno a uno, uno a muchos y opcionalidad 0..1 / 0..N).

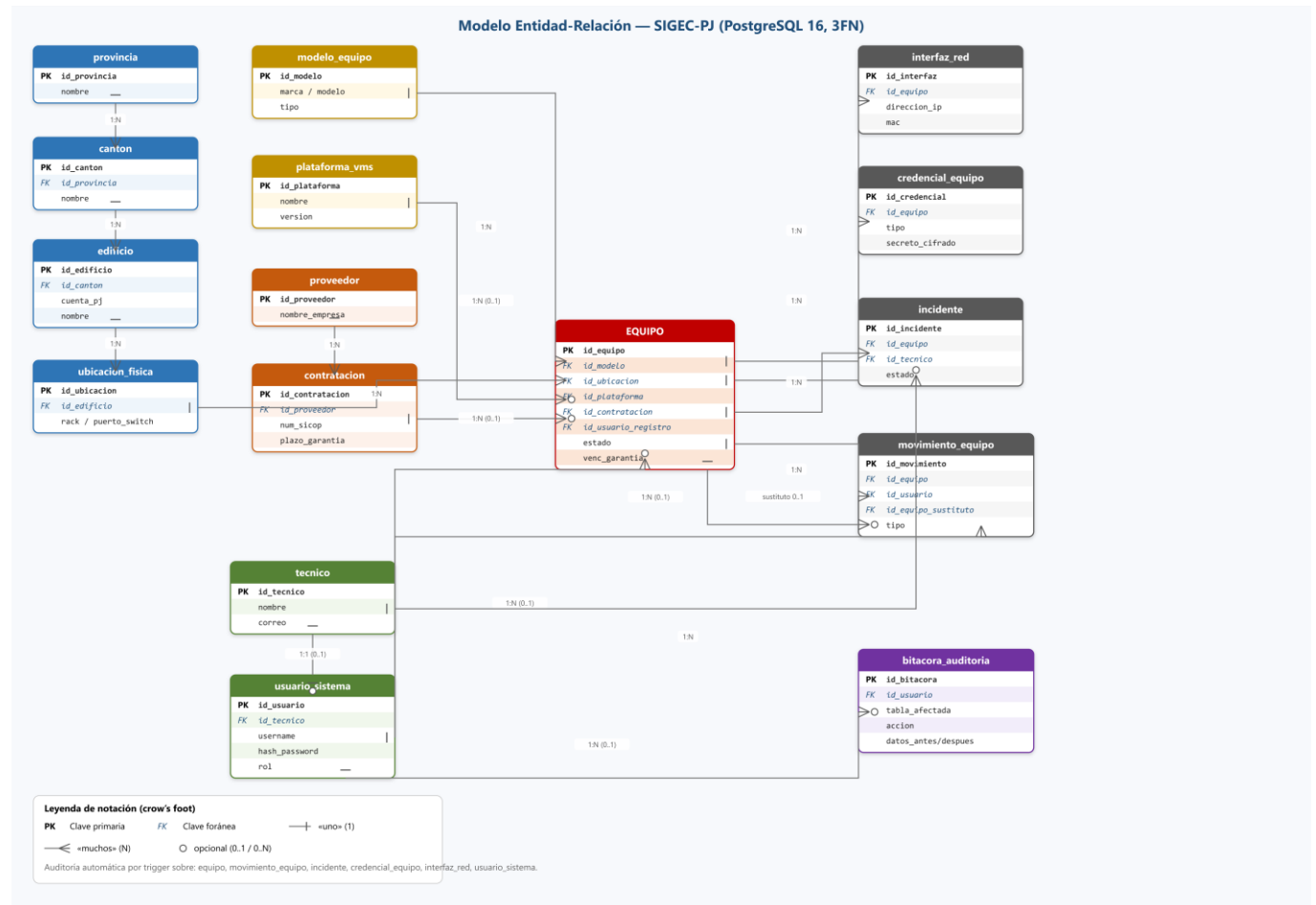


Figura 2. Modelo entidad-relación de SIGEC-PJ agrupado por capas funcionales.

### 4.2 Diccionario de datos

#### Catálogos geográficos

| Tabla     | Columna           | Tipo        | Regla            |
|-----------|-------------------|-------------|------------------|
| provincia | id_provincia (PK) | SMALLINT    |                  |
|           | nombre            | VARCHAR(40) | NOT NULL, UNIQUE |

| Tabla  | Columna           | Tipo        | Regla                             |
|--------|-------------------|-------------|-----------------------------------|
| canton | id_canton (PK)    | SERIAL      |                                   |
|        | id_provincia (FK) | SMALLINT    | NOT NULL → provincia              |
|        | nombre            | VARCHAR(60) | NOT NULL, UNIQUE con id_provincia |

### Infraestructura física

| Tabla            | Columna                      | Tipo        | Regla                                    |
|------------------|------------------------------|-------------|------------------------------------------|
| edificio         | id_edificio (PK)             | SERIAL      |                                          |
|                  | id_canton (FK)               | INT         | NOT NULL → canton                        |
|                  | cuenta_pj                    | VARCHAR(20) | NOT NULL, UNIQUE (ej. PJ-30)             |
|                  | nombre / direccion           | VARCHAR     | nombre NOT NULL                          |
| ubicacion_fisica | id_ubicacion (PK)            | SERIAL      |                                          |
|                  | id_edificio (FK)             | INT         | NOT NULL → edificio                      |
|                  | piso, rack, puerto_switch... | VARCHAR     | opcionales; precisan la ubicación exacta |

### Catálogos técnicos y contractuales

| Tabla          | Columna                 | Tipo          | Regla                                              |
|----------------|-------------------------|---------------|----------------------------------------------------|
| plataforma_vms | id_plataforma (PK)      | SMALLSERIAL   |                                                    |
|                | nombre                  | VARCHAR(40)   | NOT NULL, UNIQUE (Milestone, VX, Eclipse)          |
|                | version, puerto_defecto | VARCHAR / INT | puerto entre 1 y 65535                             |
| modelo_equipo  | id_modelo (PK)          | SERIAL        |                                                    |
|                | marca, modelo           | VARCHAR       | NOT NULL, UNIQUE(marca, modelo)                    |
|                | tipo                    | VARCHAR(12)   | CHECK IN ('CAMARA','GRABADOR','SERVIDOR','SWITCH') |
| proveedor      | id_proveedor (PK)       | SERIAL        |                                                    |
|                | nombre_empresa          | VARCHAR(150)  | NOT NULL, UNIQUE                                   |
| contratacion   | id_contratacion (PK)    | SERIAL        |                                                    |
|                | id_proveedor (FK)       | INT           | NOT NULL → proveedor                               |
|                | num_sicop, num_acta...  | VARCHAR       | trazabilidad de compras públicas                   |
|                | plazo_garantia          | INT           | meses, CHECK ≥ 0                                   |

### Personal y seguridad del aplicativo

| Tabla   | Columna                  | Tipo    | Regla         |
|---------|--------------------------|---------|---------------|
| tecnico | id_tecnico (PK)          | SERIAL  |               |
|         | nombre, correo, telefono | VARCHAR | correo UNIQUE |

| Tabla           | Columna         | Tipo        | Regla                                   |
|-----------------|-----------------|-------------|-----------------------------------------|
| usuario_sistema | id_usuario (PK) | SERIAL      |                                         |
|                 | username        | VARCHAR(40) | NOT NULL, UNIQUE                        |
|                 | hash_password   | VARCHAR(72) | NOT NULL — hash BCrypt                  |
|                 | rol             | VARCHAR(10) | CHECK IN ('ADMIN','TECNICO','CONSULTA') |
|                 | activo          | BOOLEAN     | NOT NULL DEFAULT TRUE                   |
|                 | id_tecnico (FK) | INT         | opcional → tecnico                      |

#### Entidad central: equipo

| Tabla  | Columna                          | Tipo        | Regla                                                                   |
|--------|----------------------------------|-------------|-------------------------------------------------------------------------|
| equipo | id_equipo (PK)                   | SERIAL      |                                                                         |
|        | id_modelo (FK)                   | INT         | NOT NULL → modelo_equipo                                                |
|        | id_ubicacion (FK)                | INT         | NOT NULL → ubicacion_fisica                                             |
|        | id_plataforma (FK)               | SMALLINT    | opcional → plataforma_vms                                               |
|        | id_contratacion (FK)             | INT         | opcional → contratacion                                                 |
|        | id_usuario_registro (FK)         | INT         | opcional → usuario_sistema (autoría del alta)                           |
|        | serie, num_activo                | VARCHAR     | num_activo UNIQUE                                                       |
|        | estado                           | VARCHAR(12) | DEFAULT 'ACTIVO'; CHECK IN ('ACTIVO','DANADO','RETIRADO','REEMPLAZADO') |
|        | fecha_instalacion, venc_garantia | DATE        | CHECK venc_garantia ≥ fecha_instalacion                                 |
|        | creado_en / actualizado_en       | TIMESTAMPTZ | actualizado_en se mantiene por trigger                                  |

#### Dependientes de equipo y auditoría

| Tabla             | Columna            | Tipo        | Regla                                                       |
|-------------------|--------------------|-------------|-------------------------------------------------------------|
| interfaz_red      | id_interfaz (PK)   | SERIAL      |                                                             |
|                   | id_equipo (FK)     | INT         | NOT NULL → equipo, ON DELETE CASCADE                        |
|                   | direccion_ip       | INET        | NOT NULL, UNIQUE (valida IP nativamente)                    |
|                   | mac                | MACADDR     | UNIQUE (valida MAC nativamente)                             |
| credencial_equipo | id_credencial (PK) | SERIAL      |                                                             |
|                   | id_equipo (FK)     | INT         | NOT NULL → equipo, ON DELETE CASCADE                        |
|                   | tipo               | VARCHAR(10) | CHECK IN ('SO','VMS','TOOLBOX');<br>UNIQUE(id_equipo, tipo) |
|                   | secreto_cifrado    | BYTEA       | NOT NULL — cifrado AES-256-GCM                              |
| incidente         | id_incidente (PK)  | SERIAL      |                                                             |



| Tabla              | Columna                          | Tipo             | Regla                                                       |
|--------------------|----------------------------------|------------------|-------------------------------------------------------------|
|                    | id_equipo (FK) / id_tecnico (FK) | INT              | equipo NOT NULL; tecnico opcional                           |
|                    | estado                           | VARCHAR(12)      | DEFAULT 'ABIERTO'; CHECK IN ('ABIERTO','PROCESO','CERRADO') |
| movimiento_equipo  | id_movimiento (PK)               | SERIAL           |                                                             |
|                    | id_equipo / id_usuario (FK)      | INT              | ambos NOT NULL (autoría obligatoria)                        |
|                    | tipo                             | VARCHAR(12)      | CHECK IN ('BAJA','REEMPLAZO','TRASLADO')                    |
|                    | id_equipo_sustituto (FK)         | INT              | autorreferencia; solo si tipo=REEMPLAZO; ≠ id_equipo        |
| bitacora_auditoria | id_bitacora (PK)                 | BIGSERIAL        |                                                             |
|                    | id_usuario (FK)                  | INT              | opcional (NULL = acción de sistema)                         |
|                    | tabla_afectada / id_registro     | VARCHAR / BIGINT | NOT NULL                                                    |
|                    | accion                           | VARCHAR(6)       | CHECK IN ('INSERT','UPDATE','DELETE')                       |
|                    | datos_antes / datos_despues      | JSONB            | imagen antes/después de la fila                             |

### 4.3 Integridad y automatización

- Tipos nativos con validación: INET y MACADDR rechazan valores mal formados a nivel de columna, sin validación en la aplicación.
- CHECK constraints en estados de dominio y en coherencia de fechas (garantía posterior a instalación, reparación posterior a observación).
- Autorreferencia controlada: movimiento\_equipo.id\_equipo\_sustituto solo se permite cuando tipo = REEMPLAZO y nunca apunta al mismo equipo.
- Auditoría automática por trigger (fn\_auditoria): cada INSERT/UPDATE/DELETE en tablas sensibles se registra en bitacora\_auditoria con el estado antes/después en JSONB. El usuario autenticado se propaga con SET LOCAL app.id\_usuario y el trigger lo lee con current\_setting().
- Trigger de mantenimiento: equipo.actualizado\_en se actualiza en cada UPDATE.
- Vista v\_garantias\_por\_vencer: alerta de equipos con garantía venciendo en 90 días (equipo → ubicacion\_fisica → edificio → canton → provincia); excluye equipos RETIRADOS.
- Índices de apoyo sobre las columnas más consultadas de equipo, incidente, movimiento\_equipo y bitacora\_auditoria.

### 4.4 Extracto del DDL (entidad central y auditoría)

```
CREATE TABLE equipo (
  id_equipo      SERIAL PRIMARY KEY,
  id_modelo      INT NOT NULL REFERENCES modelo_equipo(id_modelo),
  id_ubicacion   INT NOT NULL REFERENCES ubicacion_fisica(id_ubicacion),
  estado         VARCHAR(12) NOT NULL DEFAULT 'ACTIVO'
```

```
        CHECK (estado IN ('ACTIVO','DANADO','RETIRADO','REEMPLAZADO')),
    venc_garantia DATE,
    CHECK (venc_garantia IS NULL OR fecha_instalacion IS NULL
           OR venc_garantia >= fecha_instalacion)
);

CREATE TRIGGER trg_aud_equipo
AFTER INSERT OR UPDATE OR DELETE ON equipo
FOR EACH ROW EXECUTE FUNCTION fn_auditoria('id_equipo');
```

*Script completo: db/sigec\_ddl.sql (381 líneas). Datos de ejemplo: db/sigec\_seed.sql (29 equipos, incidentes, movimientos, provincias reales de Costa Rica).*

## 5. Operaciones del sistema

La API REST expone las operaciones bajo el prefijo /api/v1, agrupadas por módulo. Todos los endpoints (salvo login) exigen un JWT válido; el control de acceso por rol se aplica con @PreAuthorize en el backend.

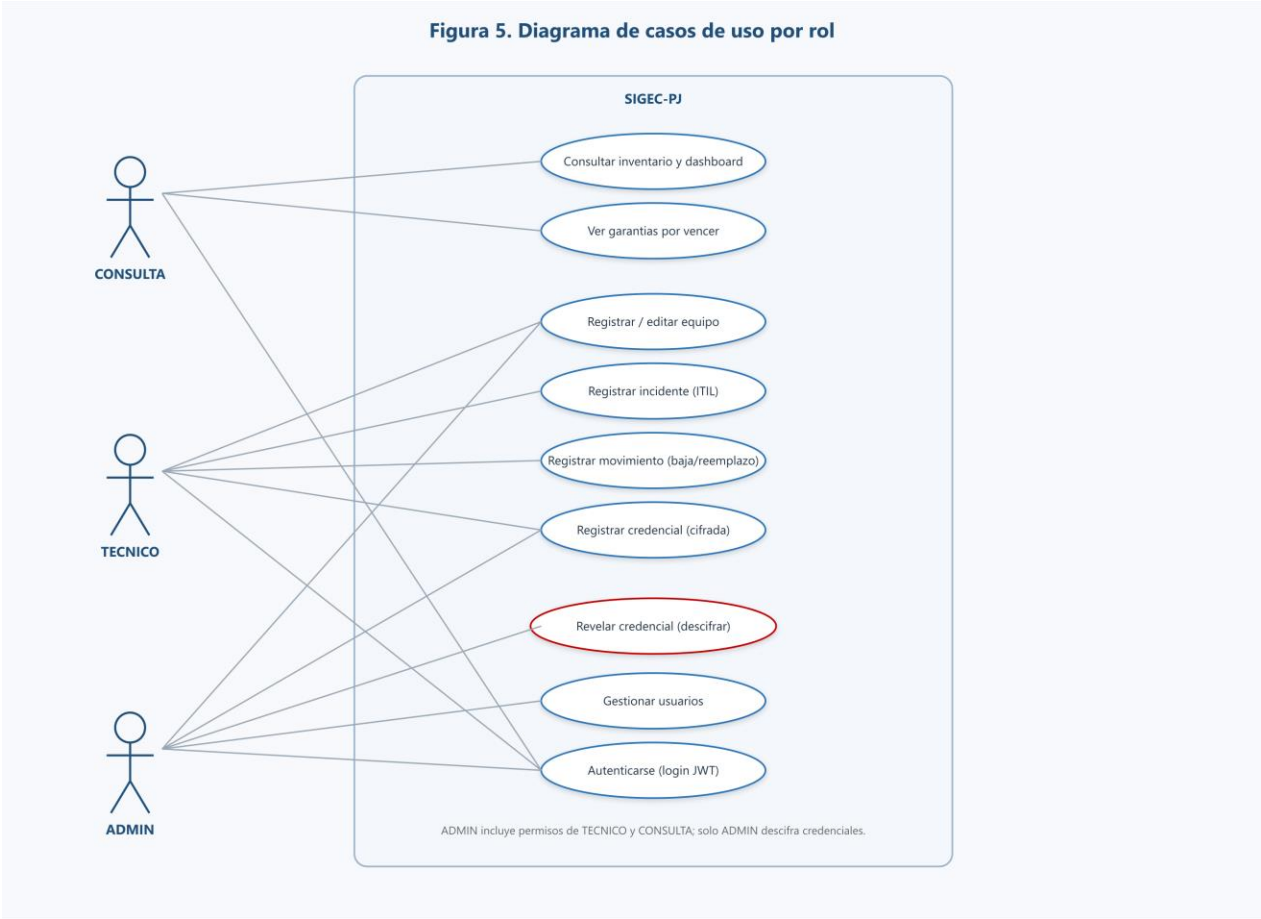


Figura 5. Casos de uso por rol (CONSULTA, TECNICO, ADMIN).

### 5.1 Autenticación

| Método | Ruta               | Descripción                                    | Acceso  |
|--------|--------------------|------------------------------------------------|---------|
| POST   | /api/v1/auth/login | Autentica y devuelve un JWT (expira en 7200 s) | Público |

### 5.2 Inventario de equipos

| Método | Ruta                                 | Descripción                                       | Acceso      |
|--------|--------------------------------------|---------------------------------------------------|-------------|
| GET    | /api/v1/equipos                      | Lista paginada con filtros (provincia, estado...) | Autenticado |
| GET    | /api/v1/equipos/garantias-por-vencer | Consulta la vista v_garantias_por_vencer          | Autenticado |
| GET    | /api/v1/equipos/{id}                 | Detalle de un equipo                              | Autenticado |

| Método | Ruta                                      | Descripción                               | Acceso         |
|--------|-------------------------------------------|-------------------------------------------|----------------|
| POST   | /api/v1/equipos                           | Alta de un nuevo equipo                   | ADMIN, TECNICO |
| PUT    | /api/v1/equipos/{id}                      | Edición de un equipo                      | ADMIN, TECNICO |
| POST   | /api/v1/equipos/{id}/interfaces           | Registra interfaz de red (IP/MAC)         | ADMIN, TECNICO |
| POST   | /api/v1/equipos/{id}/credenciales         | Registra credencial (cifrada AES-256-GCM) | ADMIN, TECNICO |
| GET    | /api/v1/equipos/credenciales/{id}/revelar | Descifra y muestra el secreto             | Solo ADMIN     |

### 5.3 Incidentes (flujo ITIL)

| Método | Ruta                    | Descripción                            | Acceso         |
|--------|-------------------------|----------------------------------------|----------------|
| GET    | /api/v1/incidentes      | Lista filtrable por estado/equipo      | Autenticado    |
| GET    | /api/v1/incidentes/{id} | Detalle de un incidente                | Autenticado    |
| POST   | /api/v1/incidentes      | Registra un incidente (estado ABIERTO) | ADMIN, TECNICO |
| PUT    | /api/v1/incidentes/{id} | Actualiza estado y trabajo realizado   | ADMIN, TECNICO |

### 5.4 Movimientos

| Método | Ruta                | Descripción                         | Acceso         |
|--------|---------------------|-------------------------------------|----------------|
| GET    | /api/v1/movimientos | Historial de movimientos            | Autenticado    |
| POST   | /api/v1/movimientos | Registra baja, reemplazo o traslado | ADMIN, TECNICO |

### 5.5 Catálogos, tablero y usuarios

Catálogos de solo lectura para formularios en cascada: /catalogos/provincias, /provincias/{id}/cantones, /cantones/{id}/edificios, /edificios/{id}/ubicaciones, /modelos, /plataformas, /contrataciones, /tecnicos. Tablero: /dashboard/resumen, /dashboard/equipos-por-provincia, /dashboard/incidentes-por-estado. Administración: /usuarios (GET y POST, solo ADMIN).

### 5.6 Ciclo de vida ITIL de un incidente

Figura 6. Ciclo de vida ITIL de un incidente

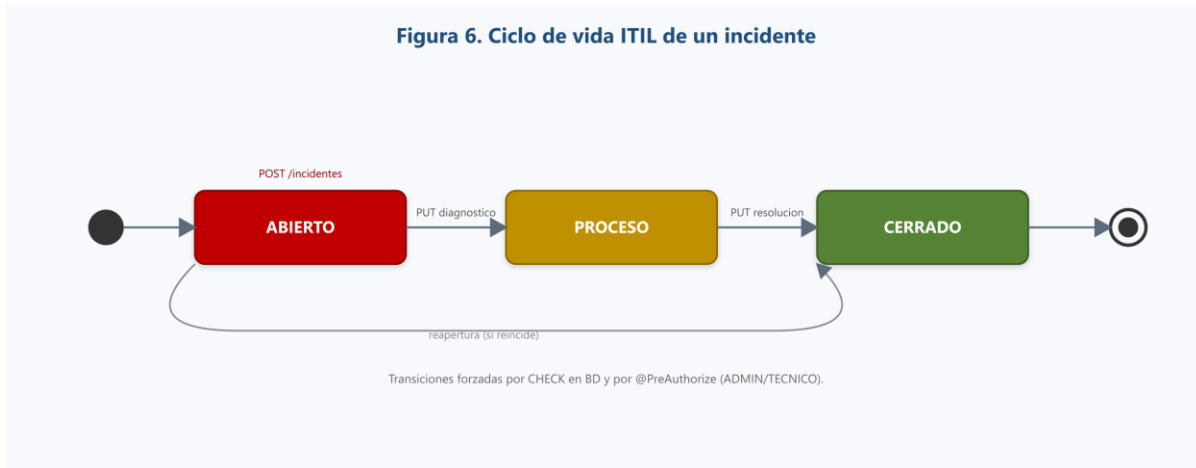


Figura 6. Estados y transiciones de un incidente.

## 5.7 Reglas de negocio clave

- Un movimiento REEMPLAZO exige un equipo sustituto distinto del original; BAJA y TRASLADO no lo permiten (regla forzada por CHECK en la base).
- Revelar una credencial es la única operación restringida exclusivamente a ADMIN; TECNICO puede registrarlas pero no descifrarlas.
- Toda alta/edición de equipo, incidente o movimiento exige un usuario autenticado, cuyo id se propaga a bitacora\_auditoria automáticamente.
- El módulo de garantías usa la vista v\_garantias\_por\_vencer en vez de calcular fechas en el backend, evitando lógica duplicada.

## 6. Seguridad

- TLS 1.3 en HAProxy para toda comunicación externa (certificado autofirmado en el laboratorio de demostración).
- Conexión JDBC backend-PostgreSQL con soporte SSL (configuración de producción).
- Credenciales de equipos CCTV cifradas con AES-256-GCM (nunca texto plano; corrige el hallazgo crítico de la Fase 1).
- Autenticación de usuarios vía JWT; contraseñas con BCrypt.
- Auditoría automática (trigger + JSONB) de todo INSERT/UPDATE/DELETE en tablas sensibles.
- Roles de base separados: postgres (administración del clúster) y sigec (aplicación, permisos acotados).
- Control de acceso por rol a nivel de endpoint (@PreAuthorize): ADMIN, TECNICO y CONSULTA.

## 7. Guía de despliegue en local

Figura 3. Diagrama de despliegue (contenedores, red y volúmenes)

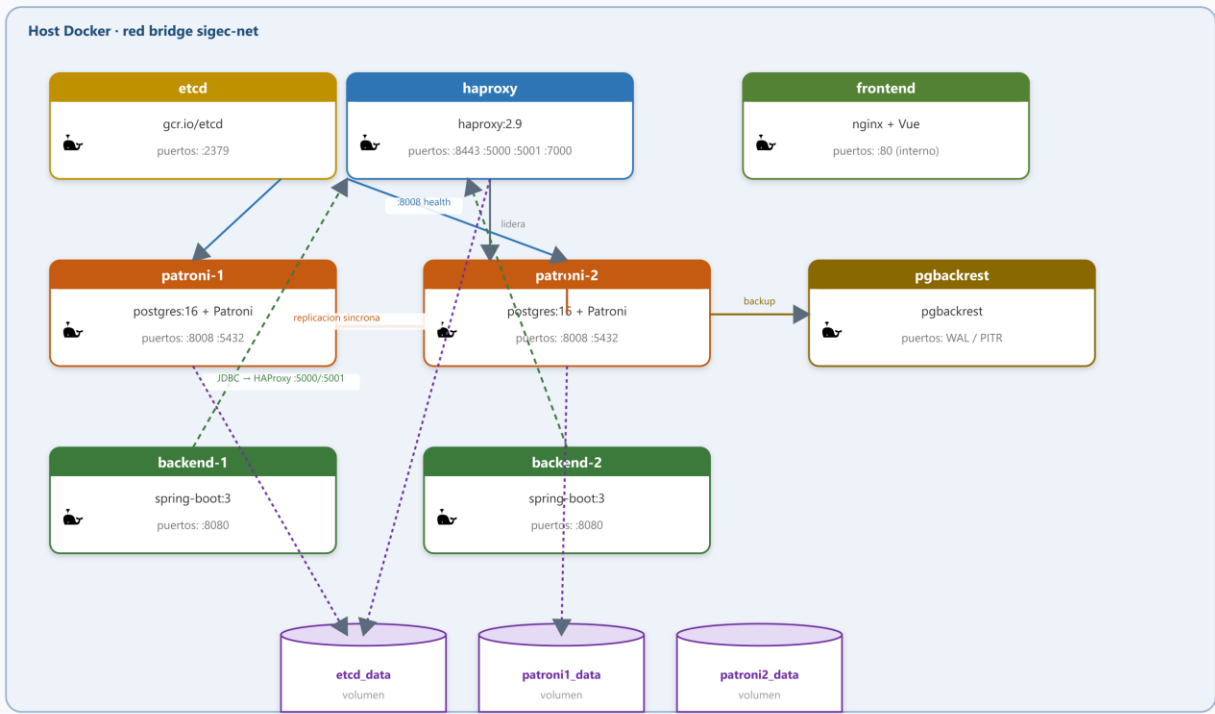


Figura 3. Despliegue en Docker: contenedores, red y volúmenes persistentes.

### 7.1 Requisitos previos

- Docker Desktop (Windows/Mac) o Docker Engine + Compose (Linux), con al menos 4 GB de RAM.
- Puertos libres en el host: 5000, 5001, 7000, 8443 y 8088 (frontend directo, opcional).
- Git para clonar el repositorio (carpeta Proyecto/) y conexión a internet en el primer arranque.

### 7.2 Variables de entorno

```
cd Proyecto/
cp .env.example .env
```

| Variable                             | Uso                                                                     |
|--------------------------------------|-------------------------------------------------------------------------|
| POSTGRES_USER /<br>POSTGRES_PASSWORD | Rol de aplicación sigec y su contraseña.                                |
| REPLICATOR_PASSWORD                  | Usuario interno de replicación entre patroni-1 y patroni-2.             |
| JWT_SECRET                           | Clave HS256 en Base64 (≥ 256 bits): openssl rand -base64 48.            |
| AES_KEY                              | Clave AES-256 en Base64 (32 bytes) para cifrar credenciales de equipos. |

## 7.3 Levantar y verificar el stack

```
docker compose up -d      # primer arranque: 2-3 min (instala Patroni vía pip)

docker exec -it sigec-patroni-1 \
  patronictl -c /etc/patroni/patroni.yml list
# + Cluster: sigec-cluster -----+
# | Member   | Role      | State   | TL | Lag |
# | patroni-1 | Leader    | running | 1  |    |
# | patroni-2 | Sync Standby | streaming | 1  | 0  |

docker compose ps        # todos 'healthy' (o 'Up' para frontend)
```

## 7.4 Acceso a la aplicación

| URL                               | Qué es                                                           |
|-----------------------------------|------------------------------------------------------------------|
| https://localhost:8443            | Aplicación web (SPA Vue.js); aceptar el certificado autofirmado. |
| https://localhost:8443/api/v1/... | API REST, misma terminación TLS.                                 |
| http://localhost:7000             | Panel de estadísticas de HAProxy (admin / admin123).             |
| Usuario inicial                   | admin / admin123 (se crea si usuario_sistema está vacía).        |

## 7.5 Problemas comunes

| Síntoma                                    | Causa típica                                   | Solución                                                            |
|--------------------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| haproxy 'unhealthy' varios minutos         | Patroni aún hace bootstrap (primer arranque)   | Esperar 2-3 min; revisar docker logs sigec-patroni-1.               |
| 'waiting for leader to bootstrap'          | Estado corrupto en etcd de un intento anterior | docker compose down + borrar volumen etcd_data + volver a levantar. |
| Backend en bucle de reinicio               | Aún no hay conexión válida a la base           | Esperar Leader + Sync Standby; el backend se recupera solo.         |
| 'password authentication failed for sigec' | El rol no se creó en el bootstrap              | Verificar post_bootstrap en ambos patroni-*.yaml (ver §8).          |
| Certificado no confiable                   | Certificado autofirmado (esperado en demo)     | Aceptar la excepción; en producción usar certificado válido.        |

## 8. Correcciones aplicadas durante la verificación

El stack de Fase 3 existía en el repositorio pero nunca se había probado de punta a punta. Al levantarlo por primera vez para este documento se encontraron y corrigieron 11 defectos reales que impedían el arranque completo:

| #  | Problema encontrado                                            | Corrección aplicada                                      |
|----|----------------------------------------------------------------|----------------------------------------------------------|
| 1  | haproxy.cfg sin salto de línea final: HAProxy no arrancaba     | Se agregó el salto de línea final.                       |
| 2  | El contenedor patroni-* nunca instalaba Patroni                | Instalación vía pip (patroni[etcd3], psycopg2-binary).   |
| 3  | Dependencia circular: HAProxy moría al no resolver DNS         | Resolución DNS diferida con resolvers + init-addr none.  |
| 4  | Faltaba la sección bootstrap: ningún nodo inicializaba         | Se agregó bootstrap.dcs + initdb + pg_hba.               |
| 5  | bin_name mal usado (debía ser bin_dir)                         | Corregido a bin_dir: /usr/lib/postgresql/16/bin.         |
| 6  | initdb intentaba correr como root y fallaba                    | gosu postgres + chown/chmod 700 del data dir.            |
| 7  | synchronous_standby_names sin comillas rompía postgresql.conf  | Reemplazado por synchronous_mode: true.                  |
| 8  | La réplica intentaba clonar vía pgBackRest sin backup previo   | Cambiado a basebackup (streaming nativo).                |
| 9  | HAProxy chequeaba salud contra el puerto 5432 (binario)        | check port 8008 (API de Patroni) + mode tcp.             |
| 10 | post_bootstrap solo en un nodo; el otro dejaba la BD sin crear | Replicado en ambos nodos + GRANT explícito al rol sigec. |
| 11 | Healthchecks rotos (nc/pg_isready)                             | bash /dev/tcp para HAProxy; pg_isready -h 127.0.0.1.     |



## 9. Evidencia de pruebas ejecutadas (Fase 3)

Todas las pruebas se ejecutaron sobre el stack levantado con docker compose up -d el 2 de julio de 2026.

### 9.1 Arranque del clúster HA

```
+ Cluster: sigec-cluster -----+
| Member   | Role       | State   | TL | Lag |
| patroni-1 | Sync Standby | streaming | 1 | 0 |
| patroni-2 | Leader     | running  | 1 |   |
# Resultado: PASA. Leader + Sync Standby, replicación lag = 0.
```

Carga del esquema y datos: verificado con \dt sobre sigecpj — las 14 entidades + bitacora\_auditoria (16 tablas). Seed: 29 equipos (cámaras, grabadores, servidores, switches) en San José y Alajuela.

### 9.2 Autenticación y API REST

```
POST /api/v1/auth/login → HTTP/1.1 200 OK
{ "token":"eyJhbGciOi...", "tipo":"Bearer",
  "expiraEnSegundos":7200, "username":"admin", "rol":"ADMIN" }

GET /api/v1/equipos (https://localhost:8443) → 200 OK (29 registros)
GET /api/v1/equipos sin token → 401 Unauthorized
```

Confirma el camino completo navegador → HAProxy(TLS) → backend Spring Boot → HAProxy(5000) → PostgreSQL primario, y que la protección JWT está activa. El frontend responde 200 OK sirviendo la SPA. Resultado: PASA.

### 9.3 Prueba de failover (RTO / RPO)

Procedimiento: con patroni-2 como líder, se detuvo su contenedor con docker stop (caída dura) y se sondeó patronictl list cada 3 s hasta ver un nuevo líder.

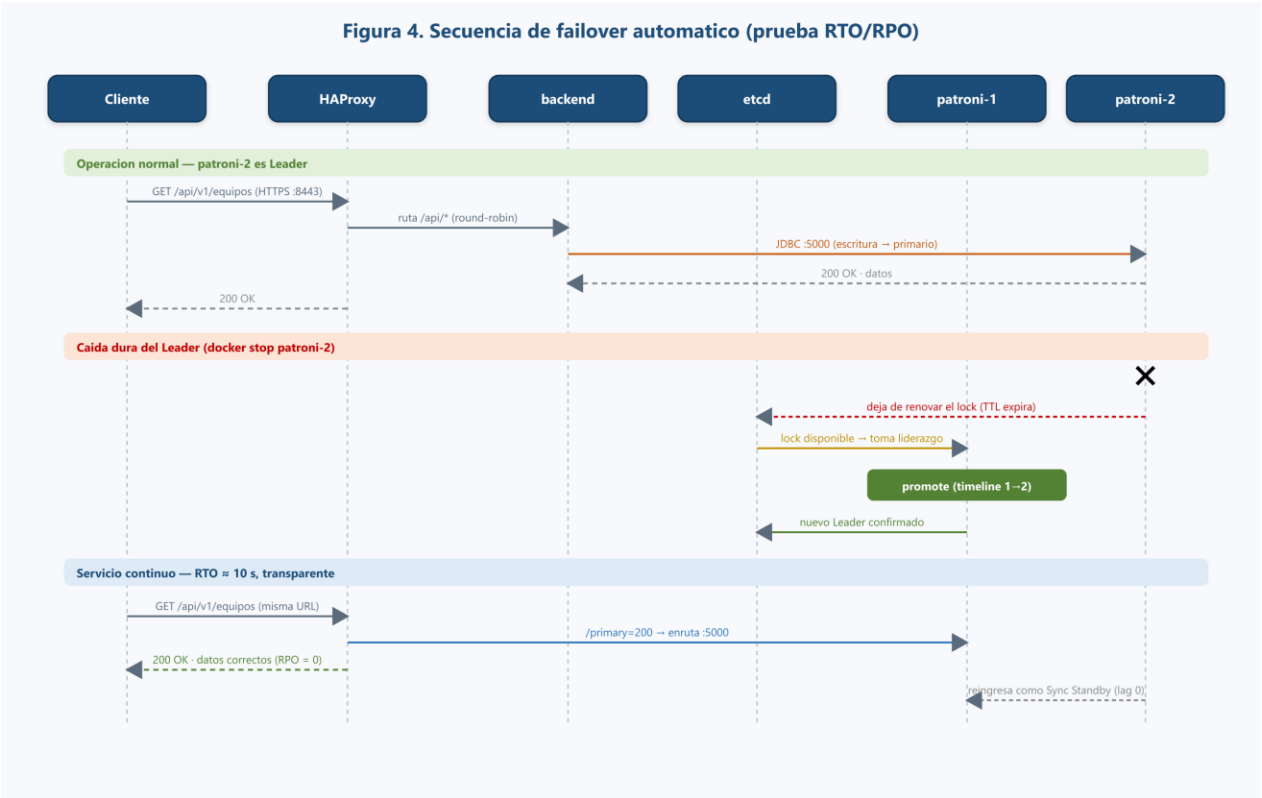


Figura 4. Secuencia observada durante el failover automático.

| Métrica                        | Resultado observado                       | Criterio                     | Cumple |
|--------------------------------|-------------------------------------------|------------------------------|--------|
| Nodo promovido                 | patroni-1 (timeline 1 → 2)                | Promoción automática         | Sí     |
| RTO (detección + promoción)    | ≈ 10 s en la corrida registrada           | RTO < 60 s                   | Sí     |
| Continuidad de servicio        | GET /equipos siguió respondiendo 200 OK   | Transparente para el cliente | Sí     |
| RPO                            | Ninguna transacción confirmada se perdió  | RPO = 0                      | Sí     |
| Reincorporación del nodo caído | patroni-2 volvió como Sync Standby, lag 0 | Reintegración automática     | Sí     |

Nota: un primer intento con docker kill no fue válido porque restart: unless-stopped revivió el contenedor antes de observar el failover. Se repitió con docker stop. Resultado: PASA — cumple los tres criterios (RTO < 60 s, RPO = 0, transparencia).

9.4 Estado final de los servicios

| Servicio                            | Estado                            |
|-------------------------------------|-----------------------------------|
| etcd                                | Up, sano                          |
| patroni-1 (PostgreSQL 16)           | Up, healthy — Leader              |
| patroni-2 (PostgreSQL 16)           | Up, healthy — Sync Standby, lag 0 |
| haproxy                             | Up, healthy                       |
| backend-1 / backend-2 (Spring Boot) | Up, healthy                       |

| Servicio               | Estado |
|------------------------|--------|
| frontend (nginx + Vue) | Up     |

## 10. Estado final y checklist de aceptación

| Criterio (definición de "hecho" de la Fase 3)                               | Verificado                                                           |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------|
| docker compose up -d levanta todo el stack sin pasos manuales               | Sí (tras las correcciones de §8)                                     |
| patronictl list muestra 1 Leader + 1 Sync Standby                           | Sí                                                                   |
| Failover automático medido, RTO < 60 s, RPO = 0                             | Sí — §9.3                                                            |
| App accesible vía HAProxy con TLS; la caída de un nodo no tumba el servicio | Sí — §9.2 y §9.3                                                     |
| Seed cargado y visible en la API/dashboard                                  | Sí — 29 equipos                                                      |
| Documentos operativos (HA, fallas, PITR, demo)                              | Presentes en docs/                                                   |
| pgBackRest / PITR                                                           | Configurado (WAL archiving y backups); procedimiento en docs/PITR.md |

*Documento generado el 2 de julio de 2026, tras levantar y probar el stack completo de SIGEC-PJ en un ambiente Docker Compose local.*